4.1 Creating own CLI script wrapping packages from python like pandas and matplotlib

Operations	Tool Uses	CLI Compatible	Action Taken
Reading CSV files	Pandas	Yes	Wrapped pandas.read_csv() into a CLI tool (read_csv.py)
Merging datasets by location	Pandas	Yes	Use pandas.merge() in merge_data.py
Testing correlations	Pandas	Yes	Use pandas.DataFrame.corr() in test_correlation.py
Comparing groups	Pandas	Yes	Use .groupby() in compare_groups.py
Visualizing results (plots)	Matplotlib	Yes	Use matplotlib in visualize.py to generate PNG plots

4.2 Missing tools

The following components required custom implementations due to project-specific logic or lack of existing CLI tools:

Operation	Tool Implementation	Descriptions
Treat missing values (-999 to NaN)	clean_missing.py	Converts placeholder values to NaN
Ensure regional allocation (Remove entries with missing location)	clean_location.py	Drops rows with null location
Save results to text format	to_text.py	Converts final data frames into .txt summaries
Full pipeline test script	main_workflow.sh	Runs all components in sequence for reproducibility

Important: In all cases, no standalone command-line tool was available that fulfilled the specific functionality we needed.

Therefore, we will create our own CLI scripts that **wrap existing Python library functions** (like pandas or matplotlib) to make them workflow-compatible.
Own implementation: we use own rules/logics; Wrapping: we use functions as intended

Abstract workflow Node	Input	Output	Implementation
Reading in both datasets	csv files	csv files	CLI script wrapping pandas.read_csv() (https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html)
Treating missing values	csv files	datasets (aus -999 -> NaN)	Own Implementation in clean_missing.py
Ensure regional allocation	datasets (without missing values)	cleaned datasets (rows with missing locations removed)	Own Implementation in clean_location.py
Merge weather and election turnout data by location	cleaned datasets	merged dataframes (with location, temperature, rain, sun, election turnout)	Own Implementation using pandas.merge() (https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.merge.html) in merge_data.py
test correlation	merged dataframes (with location, temperature, rain, sun, election turnout)	correlation coefficients, p-value	CLI script wrapping pandas.DataFrame.corr() (https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.corr.html) in test_correlation.py
Compare Groups (temperature, rain, sun)	correlation coefficients, p-value	results	CLI script wrapping pandas.DataFrame.groupby() (https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.groupby.html) in compare_groups.py
Results in text format	Data frames	results in text format	Own Implementation using built-in functions like

			DataFrame.to_string() (https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_string.html) in to_text.py
Visualization of results	Data frames	plots in PNG format	CLI script wrapping matplotlib.pyplot (https://matplotlib.org/stable/index.html) in visualize.py

Argumentation:

Reading in both datasets: CLI, because we only use an existing function read_csv() without any custom logic.
Treating missing values: Own implementation, because we apply custom rules to modify the dataset (e.g., replacing -999 with NaN).
Ensure regional allocation: Own implementation, because we apply specific filtering rules (e.g., removing rows with missing location data
Merge weather election turnout data by location: Own implementation, because we define the merging logic (columns to merge on, type of join…)
Test correlation: CLI, because we only use the existing corr() function without additional logic.
Compare Groups: CLI, because we simply apply the built-in groupby() function without modifying the logic.
Results in text format: Own implementation, because we decide on the formatting and structure of the text output ourselves.
Visualization of results: CLI, because we use existing plotting functionality from the matplotlib package without adding custom logic.